AI-Powered Ticketing and Support Assistant

Full-Stack Project Description & Development Blueprint

FRONTEND	BACKEND	AI CAPABILITIES	CLOUD & QUALITY
React	C# / ASP.NET Core	LLMs • RAG • Embeddings	Azure • Testing • CI/CD

Project summary

A full-stack support platform that applies generative AI to help support teams classify incoming tickets, summarize conversations, draft replies, search internal documentation, and identify related incidents. AI outputs remain recommendations that authorized support agents can review and approve.

Recommended build approach: deliver a working ticketing system first, add simple LLM features next, then implement retrieval-augmented generation (RAG), semantic search, testing, and cloud deployment.

Designed as a practical portfolio project for demonstrating React, .NET, SQL, AI integration, secure API design, automated testing, and Azure deployment.

1. Project Overview

The AI-Powered Ticketing and Support Assistant is a full-stack web application designed to help IT support teams manage customer issues more efficiently using artificial intelligence. Traditional ticketing workflows require agents to read incoming tickets, determine priority, assign categories, search documentation, and prepare responses manually. This project introduces AI-assisted workflows to reduce repetitive work while keeping people in control of important decisions.

When a customer submits a ticket, the system can analyze its content, recommend a category and priority, generate a concise summary, retrieve relevant troubleshooting documentation, and draft a suggested response. Agents review, edit, and approve recommendations through an interactive dashboard. The application can also identify semantically similar tickets and answer questions using approved knowledge-base documents.

Project objectives

- Reduce time spent reading and categorizing support tickets.
- Improve consistency of ticket prioritization and routing.
- Help agents respond more quickly with editable AI-generated drafts.
- Make troubleshooting documentation easier to find with semantic search.
- Identify potential duplicate tickets and recurring issues.
- Demonstrate practical use of LLMs, embeddings, RAG, and tool calling.
- Apply clean architecture, dependency injection, automated tests, access control, and cloud deployment.

2. Core Features and Functional Requirements

Module 1 — Ticket management

- Create, view, update, assign, filter, and track tickets.
- Store title, description, category, priority, status, assignee, comments, and timestamps.
- Support Open, In Progress, Resolved, and Closed statuses.
- Provide role-based access for customers, agents, and administrators.

Module 2 — Intelligent ticket classification

- Recommend a category such as Account Access, Technical, Billing, or Network.
- Recommend Low, Medium, High, or Critical priority and a suggested support team.
- Identify urgency and sentiment indicators and provide a brief explanation.
- Validate AI output and apply business rules before displaying or applying recommendations.

Module 3 — AI ticket summarization

- Summarize long ticket descriptions and comment histories.
- Highlight the main issue, troubleshooting already attempted, current state, and next steps.
- Refresh summaries when new information materially changes the ticket.

Module 4 — AI reply generation

- Draft clear, professional, empathetic responses using ticket context.
- Allow agents to edit or regenerate a draft before approval.
- Record approved or sent messages; do not send automatically without authorization.

Module 5 — Knowledge-base assistant (RAG)

- Ingest approved PDFs or text documents, split them into chunks, and generate embeddings.
- Retrieve relevant passages for a user's question and generate a grounded answer.
- Show source document names and supporting passages.
- Say when available sources are insufficient rather than inventing a solution.

Module 6 — Duplicate and related ticket detection

- Compare ticket descriptions by meaning rather than exact wording.
- Display related incidents and statuses, and suggest linking possible duplicates.
- Treat similarity as a recommendation, not proof that tickets describe the same incident.

Module 7 — AI assistant with tool calling

- Use an allowlist of backend tools to retrieve ticket details and search related incidents.
- Look up approved troubleshooting procedures and prepare proposed updates or escalations.
- Start with read-only tools; require authorization and human approval for consequential changes.

3. Recommended Technology Stack

Layer	Technology	Purpose
Frontend	React; JavaScript or TypeScript	Dashboard, forms, AI recommendations
Backend	C# / ASP.NET Core Web API	Business logic, APIs, authorization
Architecture	Clean Architecture; dependency injection	Maintainability and separation of concerns
Database	SQL Server; Entity Framework Core	Tickets, users, comments, audit records
AI integration	LLM API; Microsoft.Extensions.AI	Classification, summaries, reply generation
Embeddings	Embedding model	Semantic representation of tickets and documents
Vector search	Azure AI Search or another vector database	RAG retrieval and similarity search
Authentication	Microsoft Entra ID or ASP.NET Core Identity	Authentication and access control
Testing	xUnit; Moq or NSubstitute	Unit tests and dependency mocking
API exploration	Swagger / OpenAPI	Document and test endpoints
DevOps	GitHub Actions; Docker	Automated builds and CI/CD
Cloud	Azure App Service or Container Apps	Application hosting

Start with a hosted LLM API and a small sample document collection. Add managed vector search, advanced orchestration, and other cloud services when the core application is working.

4. High-Level System Architecture

Component	Responsibility
React web application	Ticket forms, ticket dashboard, AI recommendations, source citations, approval controls.
ASP.NET Core API	Validates requests, enforces authorization, coordinates business workflows, and returns results.
SQL Server	Persists users, tickets, comments, AI analysis metadata, and audit events.
AI application service	Builds prompts, calls the model, validates structured output, and handles provider errors.
Embedding and vector search	Indexes knowledge chunks and retrieves semantically relevant passages or similar tickets.
Knowledge documents	Approved support guides, FAQs, and troubleshooting procedures.

- A customer creates a ticket through React.
- The API validates and stores the ticket in SQL Server.
- An AI service generates structured recommendations.
- When useful, the system retrieves related tickets and knowledge passages.
- The backend validates results and displays them for agent review.
- The agent approves an action; the backend checks authorization, applies the change, and records an audit event.

5. Suggested API Design

Method	Endpoint	Purpose
POST	/api/tickets	Create ticket
GET	/api/tickets	List and filter tickets
GET	/api/tickets/{id}	Retrieve ticket details
PUT	/api/tickets/{id}	Update an authorized ticket
POST	/api/tickets/{id}/comments	Add a comment
POST	/api/ai/classify-ticket	Classify a ticket
POST	/api/ai/summarize-ticket	Summarize a ticket
POST	/api/ai/suggest-reply	Generate a reply draft
POST	/api/ai/ask-knowledge-base	Ask a document-grounded question
GET	/api/tickets/{id}/related	Find related tickets

Prefer validated request DTOs and ticket IDs over trusting arbitrary ticket content from the client. Tool execution should be internal to the backend and controlled by an allowlist, not exposed as an unrestricted command endpoint.

6. Database Design

Users — Id, Name, Email, Role

Tickets — Id, CreatedById, AssignedToId, Title, Description, Category, Priority, Status, CreatedAt

Comments — Id, TicketId, UserId, Content, CreatedAt

AIAnalyses — Id, TicketId, AnalysisType, ResultJson, ModelName, CreatedAt

AuditLogs — Id, UserId, Action, EntityType, EntityId, CreatedAt

Optional additions include KnowledgeDocuments, DocumentChunks, TicketRelationships, and AIActionApprovals. Embeddings can be stored in the vector search service with metadata rather than directly in SQL Server. Define retention policies for prompts, outputs, document indexes, and audit records, and avoid placing sensitive ticket content in general-purpose logs.

7. AI Implementation Details

Ticket classification

Send the ticket title and description to the model and request a structured result. Validate the result against an expected schema and allowed values before showing it in the UI.

```
Example response: {"category":"Account
Access","recommendedPriority":"High","sentiment":"Frustrated","suggestedTeam":"Identity
Support","reason":"The customer remains unable to log in after multiple password resets."}
```

Priority should also be checked against explicit business rules. For example, a confirmed service-wide outage may trigger a higher priority than the model recommends.

RAG knowledge assistant

- Load and clean approved documents.
- Split text into meaningful chunks and preserve source metadata.
- Generate embeddings and store them in a vector index.
- Embed the user's question and retrieve relevant chunks.
- Prompt the LLM to answer from retrieved evidence and indicate when evidence is insufficient.
- Return the answer with source names and supporting passages.

AI evaluation

Build a small evaluation set of representative tickets with expected categories, priorities, and answers. Track classification accuracy, retrieval relevance, answer quality, latency, and token cost. Compare prompt or model changes against the same test set before deployment.

8. Security, Reliability, and Responsible AI

- Enforce authentication, role-based access control, and ticket ownership checks.
- Validate all user input and model-generated output; use allowlisted values for fields such as priority and category.
- Manage secrets with secure configuration or Azure Key Vault.
- Apply rate limits, request-size limits, timeouts, and controlled retries to AI endpoints.
- Treat ticket text and retrieved documents as untrusted input to reduce prompt-injection risk.
- Require authorization and human approval for consequential actions.
- Record important approvals and changes in audit logs.
- Protect sensitive data and define retention policies for prompts, outputs, and document indexes.

- Handle provider failures gracefully and make it clear when AI recommendations are unavailable.

AI-generated content is a recommendation, not a source of authority. The model must not override permissions, reveal other customers' tickets, or execute arbitrary database queries.

9. Six-Week Development Roadmap

Phase	Focus	Deliverables
Week 1	Core ticketing	React + API setup; EF Core entities; CRUD; filtering; basic auth
Week 2	First AI features	LLM integration; classification; summaries; reply drafts
Weeks 3–4	RAG and semantic search	Document ingestion; embeddings; vector retrieval; source citations; related tickets
Week 5	Testing and hardening	xUnit tests; model-failure tests; authorization; audit logs; evaluation set
Week 6	Cloud deployment	Docker where useful; Azure deployment; secrets; monitoring; CI/CD; documentation

This is a suggested schedule, not a guarantee. If RAG, testing, or cloud deployment is new, extend the timeline rather than skipping reliability work.

10. Definition of Done

- Users can create, manage, filter, and track tickets through a working React interface.
- AI classification, summaries, and reply drafts work and display as recommendations.
- The knowledge assistant answers from sample documents and displays verifiable citations.
- Related tickets can be found using semantic similarity.
- AI provider failures and invalid outputs are handled gracefully.
- Unauthorized users cannot access restricted tickets or perform privileged actions.
- Automated tests cover core business logic, authorization, and AI failure cases.
- The repository includes setup instructions, architecture notes, sample data, and deployment guidance.

11. Portfolio and Resume Description

AI-Powered Ticketing and Support Assistant
React | C# | ASP.NET Core | SQL Server | Generative AI | RAG | Azure

Developing a full-stack AI-powered support platform that integrates large language models into ticket management workflows. Implementing intelligent ticket classification, priority recommendations, conversation summarization, and AI-generated response drafts using structured LLM outputs. Building a retrieval-augmented generation (RAG) pipeline with embeddings and vector search to answer support questions using relevant knowledge-base documents. Designing secure REST APIs, role-based access controls, and automated unit tests, with plans for Azure deployment and CI/CD integration.

Recommended implementation order

Build the core ticketing system first, then add classification and reply generation before implementing RAG. This creates a working application at each stage and demonstrates both software engineering fundamentals and practical AI integration.